

Test Cases — JSON Request/Response Blocks

Test 1: Create Event — Happy Path

Request:

```
json

POST /events
{
  "name": "Summer Conference 2026",
  "total_seats": 100,
  "event_date": "2026-08-20"
}
```

Response (201):

```
json

{
  "id": 1,
  "name": "Summer Conference 2026",
  "event_date": "2026-08-20",
  "total_seats": 100,
  "total_registrations": 0,
  "available_seats": 100
}
```

Test 2: Create Event — Duplicate Name

Request 1:

```
json

POST /events
{
  "name": "Summer Conference 2026",
  "total_seats": 100,
  "event_date": "2026-08-20"
}
```

Response 1 (201):

```
json
```

```
{
  "id": 1,
  "name": "Summer Conference 2026",
  "event_date": "2026-08-20",
  "total_seats": 100,
  "total_registrations": 0,
  "available_seats": 100
}
```

Request 2 (duplicate):

```
json

POST /events
{
  "name": "Summer Conference 2026",
  "total_seats": 100,
  "event_date": "2026-08-20"
}
```

Response 2 (400):

```
json

{
  "detail": "Event name already exists"
}
```

Test 3: Create Event — Invalid Seats (Negative)

Request:

```
json

POST /events
{
  "name": "Bad Event",
  "total_seats": -5,
  "event_date": "2026-08-20"
}
```

Response (422):

```
json
```

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "total_seats"],
      "msg": "Input should be greater than 0",
      "input": -5,
      "ctx": {"gt": 0}
    }
  ]
}
```

Test 4: Create Event — Invalid Seats (Zero)

Request:

```
json
POST /events
{
  "name": "Zero Seat Event",
  "total_seats": 0,
  "event_date": "2026-08-20"
}
```

Response (422):

```
json
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "total_seats"],
      "msg": "Input should be greater than 0",
      "input": 0,
      "ctx": {"gt": 0}
    }
  ]
}
```

Request:

```
json

POST /events
{
  "name": "Past Event",
  "total_seats": 50,
  "event_date": "2020-01-01"
}
```

Response (422):

```
json

{
  "detail": [
    {
      "type": "value_error",
      "loc": ["body", "event_date"],
      "msg": "Value error, event_date must be in the future",
      "input": "2020-01-01"
    }
  ]
}
```

Test 6: Create Event — Today's Date

Request:

```
json

POST /events
{
  "name": "Today Event",
  "total_seats": 50,
  "event_date": "2026-06-09"
}
```

Response (422):

```
json
```

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": ["body", "event_date"],
      "msg": "Value error, event_date must be in the future",
      "input": "2026-06-09"
    }
  ]
}
```

Test 7: Register User — Happy Path

Request:

```
json
POST /registrations
{
  "user_name": "Ali",
  "event_id": 1
}
```

Response (201):

```
json
{
  "id": 1,
  "event_id": 1,
  "user_name": "Ali",
  "status": "active",
  "registered_at": "2026-06-09T17:45:30"
}
```

Test 8: Register User — Duplicate Registration

Request 1:

```
json
```

```
POST /registrations
{
  "user_name": "Ali",
  "event_id": 1
}
```

Response 1 (201):

```
json
{
  "id": 1,
  "event_id": 1,
  "user_name": "Ali",
  "status": "active",
  "registered_at": "2026-06-09T17:45:30"
}
```

Request 2 (duplicate):

```
json
POST /registrations
{
  "user_name": "Ali",
  "event_id": 1
}
```

Response 2 (400):

```
json
{
  "detail": "User is already registered for this event"
}
```

Test 9: Register User — Event Full

Request:

```
json
```

```
POST /registrations
{
  "user_name": "Charlie",
  "event_id": 2
}
```

Response (400):

```
json
{
  "detail": "Event is full"
}
```

Test 10: Register User — Non-Existent Event

Request:

```
json
POST /registrations
{
  "user_name": "Ali",
  "event_id": 99999
}
```

Response (404):

```
json
{
  "detail": "Event not found"
}
```

Test 11: View Events — All Events

Request:

```
GET /events
```

Response (200):

json

```
[
  {
    "id": 1,
    "name": "Event 1",
    "event_date": "2026-08-15",
    "total_seats": 50,
    "total_registrations": 10,
    "available_seats": 40
  },
  {
    "id": 2,
    "name": "Event 2",
    "event_date": "2026-12-25",
    "total_seats": 100,
    "total_registrations": 50,
    "available_seats": 50
  },
  {
    "id": 3,
    "name": "Event 3",
    "event_date": "2026-10-31",
    "total_seats": 20,
    "total_registrations": 20,
    "available_seats": 0
  }
]
```

Test 12: View Events — Filter Upcoming Only

Request:

```
GET /events?upcoming_only=true
```

Response (200):

json


```
[
  {
    "id": 2,
    "name": "Event 2",
    "event_date": "2026-08-15",
    "total_seats": 100,
    "total_registrations": 0,
    "available_seats": 100
  },
  {
    "id": 3,
    "name": "Event 3",
    "event_date": "2026-12-25",
    "total_seats": 50,
    "total_registrations": 0,
    "available_seats": 50
  }
]
```

Test 13: View Events — Sort by Date

Request:

```
GET /events?sort_by_date=true
```

Response (200):

```
json
```

```
[
  {
    "id": 2,
    "name": "Event B",
    "event_date": "2026-08-15",
    "total_seats": 100,
    "total_registrations": 0,
    "available_seats": 100
  },
  {
    "id": 3,
    "name": "Event C",
    "event_date": "2026-10-31",
    "total_seats": 50,
    "total_registrations": 0,
    "available_seats": 50
  },
  {
    "id": 1,
    "name": "Event A",
    "event_date": "2026-12-25",
    "total_seats": 75,
    "total_registrations": 0,
    "available_seats": 75
  }
]
```

Test 14: View Events — Filter & Sort Combined

Request:

```
GET /events?upcoming_only=true&sort_by_date=true
```

Response (200):

```
json
```

```
[
  {
    "id": 3,
    "name": "Event 3",
    "event_date": "2026-08-15",
    "total_seats": 100,
    "total_registrations": 0,
    "available_seats": 100
  },
  {
    "id": 2,
    "name": "Event 2",
    "event_date": "2026-12-25",
    "total_seats": 50,
    "total_registrations": 0,
    "available_seats": 50
  }
]
```

Test 15: Cancel Registration — Happy Path

Request:

```
DELETE /registrations/1/Ali
```

Response (200):

```
json
{
  "message": "Registration cancelled successfully",
  "event_id": 1,
  "user_name": "Ali"
}
```

Test 16: Cancel Registration — Already Cancelled

Request:

```
DELETE /registrations/1/Ali
```

Response (404):

```
json
{
  "detail": "No active registration found"
}
```

Test 17: Cancel Registration — Non-Existent User

Request:

```
DELETE /registrations/1/NonExistentUser
```

Response (404):

```
json
{
  "detail": "No active registration found"
}
```

Test 18: Seat Count Accuracy After Multiple Operations

Operation 1 - Register User1:

```
json
POST /registrations
{
  "user_name": "User1",
  "event_id": 1
}
```

Response (201):

```
json
```

```
{
  "id": 1,
  "event_id": 1,
  "user_name": "User1",
  "status": "active",
  "registered_at": "2026-06-09T17:45:30"
}
```

Check available seats:

```
GET /events
```

Response includes:

```
json
{
  "id": 1,
  "total_seats": 10,
  "total_registrations": 1,
  "available_seats": 9
}
```

Operation 2 - Register User2:

```
json
POST /registrations
{
  "user_name": "User2",
  "event_id": 1
}
```

Check available seats: (available_seats should be 8)

Operation 3 - Register User3:

```
json
POST /registrations
{
  "user_name": "User3",
  "event_id": 1
}
```

Check available seats: (available_seats should be 7)

Operation 4 - Cancel User2:

```
DELETE /registrations/1/User2
```

Response (200):

```
json
{
  "message": "Registration cancelled successfully",
  "event_id": 1,
  "user_name": "User2"
}
```

Check available seats: (available_seats should be 8)

Operation 5 - Register User4:

```
json
POST /registrations
{
  "user_name": "User4",
  "event_id": 1
}
```

Check available seats: (available_seats should be 7)

Final check:

```
GET /events
```

Response includes:

```
json
{
  "id": 1,
  "total_seats": 10,
  "total_registrations": 3,
  "available_seats": 7
}
```

Test 19: Race Condition — Concurrent Registrations

Thread 1 Request:

```
json

POST /registrations
{
  "user_name": "User1",
  "event_id": 1
}
```

Thread 1 Response (201):

```
json

{
  "id": 1,
  "event_id": 1,
  "user_name": "User1",
  "status": "active",
  "registered_at": "2026-06-09T17:45:30"
}
```

Thread 2 Request (concurrent):

```
json

POST /registrations
{
  "user_name": "User2",
  "event_id": 1
}
```

Thread 2 Response (400):

```
json

{
  "detail": "Event is full"
}
```

Final check:

```
GET /events/1
```

Response includes:

json

```
{
  "id": 1,
  "total_seats": 1,
  "total_registrations": 1,
  "available_seats": 0
}
```

Test 20: Duplicate Request Handling

Request 1:

json

```
POST /registrations
{
  "user_name": "Ali",
  "event_id": 1
}
```

Response 1 (201):

json

```
{
  "id": 1,
  "event_id": 1,
  "user_name": "Ali",
  "status": "active",
  "registered_at": "2026-06-09T17:45:30"
}
```

Request 2 (sent immediately after, duplicate):

json

```
POST /registrations
{
  "user_name": "Ali",
  "event_id": 1
}
```

Response 2 (400):


```
json
```

```
{  
  "detail": "User is already registered for this event"  
}
```

Test 21: Export Events to CSV

Request:

```
GET /export/events
```

Response (200):

```
Content-Type: text/csv  
Content-Disposition: attachment; filename=events.csv  
  
id,name,event_date,total_seats,total_registrations,available_seats  
1,Event1,2026-08-15,50,10,40  
2,Event2,2026-12-25,100,50,50
```

Test 22: Export Registrations by Event

Request:

```
GET /export/registrations/1
```

Response (200):

```
Content-Type: text/csv  
Content-Disposition: attachment; filename=registrations_1.csv  
  
id,event_id,user_name,status,registered_at  
1,1,Ali,active,2026-06-09T17:45:30  
2,1,Bob,active,2026-06-09T17:46:15  
3,1,Charlie,cancelled,2026-06-09T17:47:00
```

Quick Reference: HTTP Status Codes

Code	Meaning	Used In Tests
200	OK	Cancel registration, export endpoints
201	Created	Create event, register user
400	Bad Request	Duplicate event/registration, event full
404	Not Found	Non-existent event/registration
422	Unprocessable Entity	Invalid input (Pydantic validation)